

Потоки

(которые `stream`, не `thread`)

Поток - абстрактный канал, по которому данные "текут" либо в программу, либо из программы.

При этом то, куда "текут" данные может быть чем угодно, поток лишь предоставляет интерфейс.

Стандартные потоки ввода-вывода

- `istream`
 - **`stdin`** - ввод данных в программу
- `ostream`
 - **`stdout`** - вывод результатов программы
 - **`stderr`** - вывод ошибок
 - **`stdlog`** - вывод логов

Операции ввода

1. `cin >> var` - читает до пробела
2. `getline(cin, str)` - читает всю строку до `\n`
3. `cin.get()` - читает 1 символ
4. `cin.ignore(n, \n)` - игнорирует символ
5. `cin >> std::ws` - очищает буфер

Операции вывода

1. `cout << "smth"`
2. `<< endl`
3. `cout.flush()` - очищает буфер

4. `cout.put()` - ВЫВОДИТ ОДИН СИМВОЛ

5. `setw` - задает ширину вывода

Работа с файлами

```
#include <fstream>

int main()
{
    // если файл есть - перезапишет, если нет -
    создаст
    std::ofstream file{"filename.txt"};

    // пишем данные в файл
    file << "Random text";

    // обязательно нужно ручками закрыть поток
    // если не закрыть, то другие потоки для этого
    файла не смогут с ним работать
    file.close();
}
```

```
#include <fstream>

int main()
{
    int a, b;

    /* data.txt выглядит так:
       1 2
    */

    // открываем файл
    std::ifstream file{"data.txt"};
```

```
// В a, b пишем данные из файла
file >> a >> b;

file.close();
}
```

В примерах выше есть ряд проблем, поэтому стоит себя обезопасить при работе

Безопасное открытие файла:

```
std::ifstream file {"filename.txt"};

if (!file.is_open()) {
    // если не открыли файл, то падаем с ошибкой
    std::cerr << "Err!";
    return 1;
}

// если поток (выполнения) дошел сюда, то все хорошо
...
```

Дополнительно

Добавление данных в файл, без его затирания

```
std::ofstream file{"filename.txt"};

file << "1\n";

file.close();

file.open("filename.txt", std::ios::app)
file << "2";
```

```
file.close();

/* В filename.txt будет следующее
    1
    2
*/
```

Чтение произвольного количества данных

```
int x;
int res = 0;

std::ifstream file{"data.txt"};
/* В data.txt следующие данные
    1 2 3 4 5 6 7 8 9
*/

// Вариант 1
while (file >> x) {
    // пока можем читать
    res += x
}

// Вариант 2
while (!file.eof()) { //eof - end of file
    // пока не конец файла
    ...
}

file.close();

std::cout << res << std::endl; // 45
```

Обертка для работы с любым потоком

```
int sum_from_stream(std::istream& stream)
{
    int x;
    int sum = 0;

    while (stream >> x) {
        sum += x;
    }

    return sum;
}

int main()
{
    // Для файла
    std::ifstream file{"file.txt"};

    std::cout << sum_from_stream(file);

    file.close();

    // Для стандартного потока
    std::cout << sum_from_stream(std::cin);

}
```

Шаблоны

Обобщенное программирование - метод разработки, при котором алгоритмы и структуры пишут в наиболее общем виде, без привязки к конкретным типам

Это позволяет

1. Написать код 1 раз

2. Переиспользовать для разных типов

Шаблоны функций - для создания универсальных алгоритмов.

Шаблоны классов - для создание универсальных структур.

Например, мы хотим написать функцию для замены значений между переменными

Можно сделать так:

```
void swap(int& a, int& b)
{
    int tmp = a;
    a = b;
    b = tmp;
}

void swap(double& a, double& b)
{
    double tmp = a;
    a = b;
    b = tmp;
}
```

Пока все хорошо, но если у нас еще 10 типов?

```
// T - какой-то тип
template <typename T>
void swap(T& a, T& b)
{
    T temp = a;
    a = b;
    b = temp;
}
```

Шаблоны функций - механизм, который позволяет писать обобщенный код, работающий с разными типами без дублирования.